
Group Relative Policy Optimization for Control

Jannik Busse Guldmand
MSc Data Science (ICT Systems)
Department of Mathematics and Computer Science
University of Southern Denmark
jagul24@student.sdu.dk

DM887 Reinforcement Learning
Term Project: GRPO for Control
Spring 2026
Supervisor: Professor Melih Kandemir
https://github.com/guldmand/DM887_Project

Abstract

This report studies Group Relative Policy Optimization (GRPO) for control. The full report will present a GRPO variant for continuous and discrete control tasks and evaluate it against PPO, SAC, and TD3 on Car Racing, cartpole-swingup-v0, and acrobot-swingup-v0. The midway version focuses on the required interim components: related work, MDP notation, and baseline results for PPO, SAC, and TD3.

1 Introduction

Reinforcement learning for control aims to learn decision-making policies through interaction with dynamic environments. In such settings, it is not sufficient for a method to perform well in a single run; evaluation should also be reproducible, comparable, and robust across environments, training budgets, and random seeds. This project investigates how Group Relative Policy Optimization (GRPO), originally developed in the context of language-model reinforcement learning and mathematical reasoning, can be adapted and evaluated in control-oriented benchmark environments [Shao et al., 2024].

The project uses PPO, SAC, and TD3 as baseline algorithms. These methods represent central approaches in modern deep reinforcement learning: PPO is an on-policy policy-gradient method based on stabilized policy updates, SAC is an off-policy maximum-entropy actor-critic method, and TD3 is a deterministic actor-critic method designed for continuous action spaces [Schulman et al., 2017, Haarnoja et al., 2018, Fujimoto et al., 2018]. In this report, these algorithms are not the proposed methodological contribution. Instead, they define the comparison surface needed to evaluate a later GRPO-based control method.

GRPO is relevant because it constructs policy updates from group-relative signals rather than relying solely on a conventional value-function baseline. However, transferring this idea to control is not direct. Control environments require sequential credit assignment, handling of continuous or structured action spaces, and stable learning from trajectories whose outcomes may depend strongly on the environment, seed, and training budget. A GRPO variant for control therefore requires both a precise formal problem formulation and a reliable baseline foundation.

At the midway stage, this report does not present a completed GRPO-control algorithm. The contribution is instead to establish the experimental and methodological foundation for the final comparison. The report introduces the Markov decision process notation, documents the baseline implementation,

and validates a complete experimental matrix with three algorithms, three environments, and five seeds, corresponding to 45 runs and 900 evaluation rows. The vector-based control environments are run through ObjectRL, while CarRacing is handled by project-side CNN implementations because the image observations used in this setup do not fit directly into ObjectRL’s standard vector-observation workflow [Baykal et al., 2025, Towers et al., 2024].

The results should therefore be interpreted as a midway foundation rather than as final performance conclusions. No hyperparameter optimization was performed, and the training budgets were chosen to validate the pipeline and comparison setup rather than to establish asymptotic performance. The results primarily show that the environment adapters, training scripts, evaluation protocol, result files, and figure generation work end-to-end. This provides the necessary basis for the later GRPO evaluation.

The remainder of the report is structured as follows. Section ?? reviews related work on PPO, SAC, TD3, and GRPO. Section ?? describes the formal MDP notation and experimental methodology. Section ?? outlines the theoretical and notational basis for the later GRPO extension. Section 5 presents the completed midway baseline experiments. Finally, Section ?? summarizes the current limitations and the next steps toward the final GRPO-based control method.

2 Related Work

3 Methodology

4 Theory

5 Experiments

The purpose of the midway experiments is to establish a reproducible baseline foundation for the later GRPO-control comparison. At this stage, the final GRPO-based control method has not yet been implemented. The experimental section therefore focuses on completing, validating, and interpreting the required PPO, SAC, and TD3 baselines on the three project environments.

The experiments serve as an end-to-end validation of the project pipeline. The environments must be initialized correctly, policies must be trainable, evaluations must be performed at fixed intervals, result files must be written in a consistent format, and learning curves must be generated from the stored evaluation data. Thus, the midway experiments do not only report preliminary performance; they also verify that the experimental infrastructure is ready for the final GRPO extension.

The results should be interpreted cautiously. They were produced under a limited midway training budget and without hyperparameter optimization. They should therefore not be read as final claims about the general performance of the algorithms, but as preliminary baseline results under a concrete and reproducible experimental setup.

5.1 Experimental Matrix

The midway experiment matrix consists of three algorithms, three environments, and five random seeds. The algorithms are PPO, SAC, and TD3. The environments are `cartpole_swingup`, `acrobot_swingup`, and `car_racing_continuous`. The seeds are 0, 1, 2, 3, 4. This gives

$$3 \text{ algorithms} \times 3 \text{ environments} \times 5 \text{ seeds} = 45 \text{ runs.}$$

The final report-facing notebook validates that all 45 expected algorithm–environment–seed combinations are present, and that the result set contains 900 evaluation rows. All evaluation rows have status `completed`, and no expected combination is missing. This validation is important because the reported figures and tables are therefore based on a complete midway baseline matrix rather than a partial subset of runs.

5.2 Environments and Implementation Paths

The experiments use two implementation paths because the environments have different observation types. The environments `cartpole_swingup` and `acrobot_swingup` are treated as vector-control

environments and are executed through ObjectRL. The project uses a project-side environment bridge so that the selected environments can be used with ObjectRL without modifying the external ObjectRL source code [Baykal et al., 2025].

The environment `car_racing_continuous` differs from the two vector-control environments because it uses image observations. It is therefore handled by a separate project-side CNN implementation. PPO, SAC, and TD3 are still evaluated on CarRacing, but with CNN-based policies or actors that can process image observations. The CarRacing runs were executed on Google Colab with CUDA and then copied back into the local repository, so that they are stored in the same result structure as the remaining baseline results.

This split is a practical implementation choice at the midway stage. The central point is that all three algorithms are evaluated on all three environments, and that the outputs are collected into a common result pipeline.

5.3 Evaluation Protocol

The learned policies are evaluated using undiscounted evaluation episode return. This follows the project specification, where learning curves report training steps before evaluation on the horizontal axis and undiscounted evaluation return on the vertical axis.

It is useful to distinguish the training formulation from the evaluation metric. In reinforcement learning, the objective is often formulated in terms of discounted return, for example

$$V_{\gamma}^{\pi}(s) = \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right].$$

In the experiment section, however, the reported metric is undiscounted evaluation episode return. The results therefore describe observed episode return during evaluation, not training loss or classification accuracy.

The vector-control runs are evaluated over a midway budget of 20,000 training steps, while the CarRacing runs are evaluated over 10,000 training steps. These budgets are sufficient to validate the pipeline and produce preliminary learning curves, but they are not intended to establish converged or strongly optimized policies. No hyperparameter optimization was performed; the algorithms should therefore be understood as baseline configurations.

5.4 Result Validation

Before interpreting the results, the final notebook validates the result files. The validation confirms that 45 out of 45 expected CSV files are present, that 900 out of 900 expected evaluation rows are present, that PPO, SAC, and TD3 are all represented, that all three environments are represented, and that seeds 0 through 4 are present. It also confirms that there are no missing algorithm–environment–seed combinations and that all evaluation rows have status `completed`.

This is a central midway result in itself. It shows that the baseline experiments have been completed as a full matrix, and that the report figures and tables are generated from a validated data basis.

5.5 Baseline Results

The learning curves show different patterns across the three environments. The results should be read as descriptive midway results rather than final conclusions about the general quality of the algorithms.

For `cartpole_swingup`, the learning signal is the clearest. SAC obtains the highest mean final return at the midway budget, followed by TD3. Both algorithms show clear positive development from the first to the final evaluation point. PPO ranks lower at the final evaluation point, but it is still included as a completed baseline in the same experimental matrix.

For `acrobot_swingup`, the results are more modest. PPO obtains the highest mean final return at the midway budget, followed by TD3 and SAC. PPO and TD3 show positive development over the training run, while SAC does not show the same improvement under the current short setup. This indicates that the environment is more challenging under the selected budget.

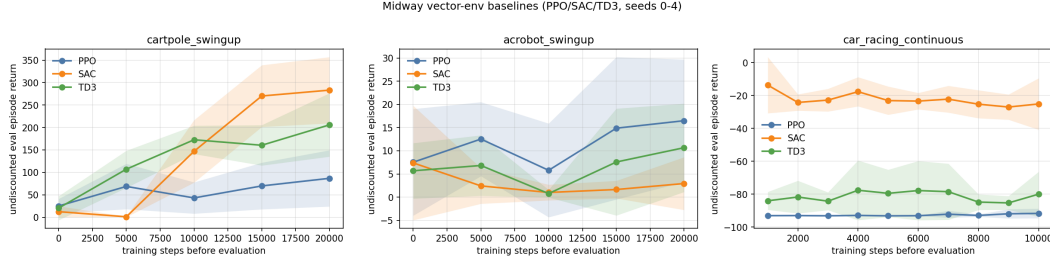


Figure 1: Midway PPO, SAC, and TD3 baseline learning curves for the vector-control environments.

For `car_racing_continuous`, the final returns remain negative for all three algorithms. This is expected under a short CNN-based midway budget. SAC obtains the best mean final return among the CarRacing baselines, followed by TD3 and PPO. The most important result for CarRacing at this stage is not high performance, but that the CNN pipeline runs successfully for all three algorithms and that CarRacing is now included in the same validated baseline matrix as the vector-control environments.

Overall, the results show that the baseline pipeline is functional and complete. There is a clear learning signal in parts of the experiment set, especially for `cartpole_swingup`, while `acrobot_swingup` and `car_racing_continuous` remain more difficult under the midway budget.

5.6 Stability and Variation

The results also show variation across seeds and evaluation points. This is expected in reinforcement learning, where initialization, exploration, and environment interaction can affect the learning trajectory.

The stability summaries should not be used to identify a generally most stable algorithm across all tasks. They describe the variation observed in the specific midway runs. An algorithm may have low variation but also low performance, while another algorithm may obtain higher final return with greater variation.

For example, SAC obtains the best final return on CarRacing at the midway budget, but it is not necessarily the least variable method. On CartPole, SAC and TD3 obtain stronger final performance than PPO, but with more variation. This supports the overall interpretation: the results are useful for validating the pipeline and identifying preliminary patterns, but they are not sufficient for final claims about robustness or generalization.

5.7 Limitations

The midway experiments have four main limitations. First, the training budgets are short. The results therefore show early learning behaviour rather than necessarily converged policies. This is particularly relevant for CarRacing, where image observations and CNN-based policy learning typically require longer training.

Second, no hyperparameter optimization was performed. Differences between PPO, SAC, and TD3 may therefore reflect both algorithmic differences and the selected baseline configurations.

Third, the GRPO-control method has not yet been implemented in these experiments. This section therefore does not evaluate the final proposed method. Instead, it establishes the baseline foundation against which the final method should be compared.

Fourth, the project uses two implementation paths: ObjectRL for vector-control environments and project-side CNN agents for CarRacing. This is a practical choice at the midway stage, but it also means that differences between vector-control and CarRacing reflect not only environment difficulty, but also differences in observation type and model architecture.

5.8 Transition to the Final GRPO Stage

The main conclusion from the experiments is that the baseline foundation is now in place. All PPO, SAC, and TD3 runs have been completed on all three environments with five seeds, and the results are collected in a validated result pipeline.

In the final project stage, the same experimental structure should be reused for GRPO-control. GRPO should be evaluated on the same environments, with the same seed structure and the same evaluation metric. This will make the final comparison between GRPO, PPO, SAC, and TD3 more consistent and reproducible.

The midway experiments therefore do not answer the full research question of the project. Instead, they establish the infrastructure, baseline performance, and result validation on which the final GRPO analysis depends.

6 Conclusion

References

- Gulcin Baykal, Abdullah Akg"ul, Manuel Haussmann, Bahareh Tasdighi, Nicklas Werge, Yi-Shan Wu, and Melih Kandemir. Objectrl: An object-oriented reinforcement learning codebase. *arXiv preprint arXiv:2507.03487*, 2025. URL <https://arxiv.org/abs/2507.03487>.
- Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, 2018. URL <https://arxiv.org/abs/1802.09477>.
- Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018. URL <https://arxiv.org/abs/1812.05905>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Mark Towers, Ariel Kwiatkowski, Jordan K. Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Arjun KG, Markus Krimmel, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL <https://arxiv.org/abs/2407.17032>.

A Proofs